Методические указания к лабораторной работе

Автор: доцент НИТПУ Друки А.А.

Распознавание объектов на изображениях

Данная лабораторная работа посвящена получению навыков работы с искусственными нейронными сетями (НС), изучению функционала библиотеки машинного обучения Keras, применение НС в задачах компьютерного зрения.

Цель работы: получить навыки распознавания объектов на изображениях с использованием искусственных нейронных сетей в библиотеке машинного обучения Keras. В качестве объектов на изображениях будут использоваться изображения элементов одежды Fashion MNIST.

Задачи:

1. Реализация топологии НС;
2. Подготовка данных для обучения НС;
3. Реализация топологии НС;
4. Обучение НС;
5. Тестирование НС;
6. Расчеты на основе обученной НС.

Классификация изображений

Классификация изображений – один из разделов машинного обучения, посвященный извлечению классов информации из растрового изображения. На растровых изображениях имеется множество объектов, которые можно отнести к разным классам по определенным характеристикам. В процессе классификации происходи извлечение ключевых характеристик на изображении и отнесение исследуемых объектов на изображении к тем, или иным классам.

Библиотека Keras

Keras – открытая нейросетевая библиотека, написанная на языке Python. Официальный сайт: https://keras.io/ Нацелена на оперативную работу с нейронными сетями, при этом обладает компактностью, модульностью и расширяемостью. Представляет собой надстройку над более низкоуровневыми фреймворками TensorFlow и Theano.

Библиотека Keras следует лучшим методам снижения когнитивной нагрузки: он предлагает последовательные и простые API-интерфейсы (программный интерфейс приложения, интерфейс прикладного программирования), сводит к минимуму количество действий пользователя, необходимых для типичных случаев использования, и предоставляет четкие и действенные сообщения об ошибках. Он также содержит обширную документацию и руководства для разработчиков.

Данная библиотека используется CERN, NASA, NIH и многими другими научными организациями для решения задач по машинному обучению во всем мире. Keras обладает низкоуровневой гибкостью для реализации произвольных исследовательских идей, предлагая дополнительные высокоуровневые удобные

функции для ускорения циклов экспериментов. Благодаря простому использованию и ориентированию на пользователя, Keras является предпочтительным решением для глубокого обучения для многих университетских курсов. Он широко рекомендуется как один из лучших способов изучения глубокого обучения.

Ход работы

1. Загрузка данных

В качестве изображений будет использоваться база изображений с элементами одежды Fashion MNIST (Modified National Institute of Standards and Technology). MNIST является стандартом, предложенным Национальным институтом стандартов и технологий США с целью калибрации и сопоставления методов распознавания изображений с помощью машинного обучения в первую очередь на основе нейронных сетей. Данные состоят из заранее подготовленных примеров изображений, на основе которых проводится обучение и тестирование систем. Выборка разбита на 10 классов изображений: футболка, штаны, кофта, платье, пальто, сандалии, рубашка, кроссовки, сумка, ботильоны. Данная выборка содержит 60000 изображений размером 28x28 пикселей. На рисунке 1 представлены примеры изображений базы Fashion MNIST.

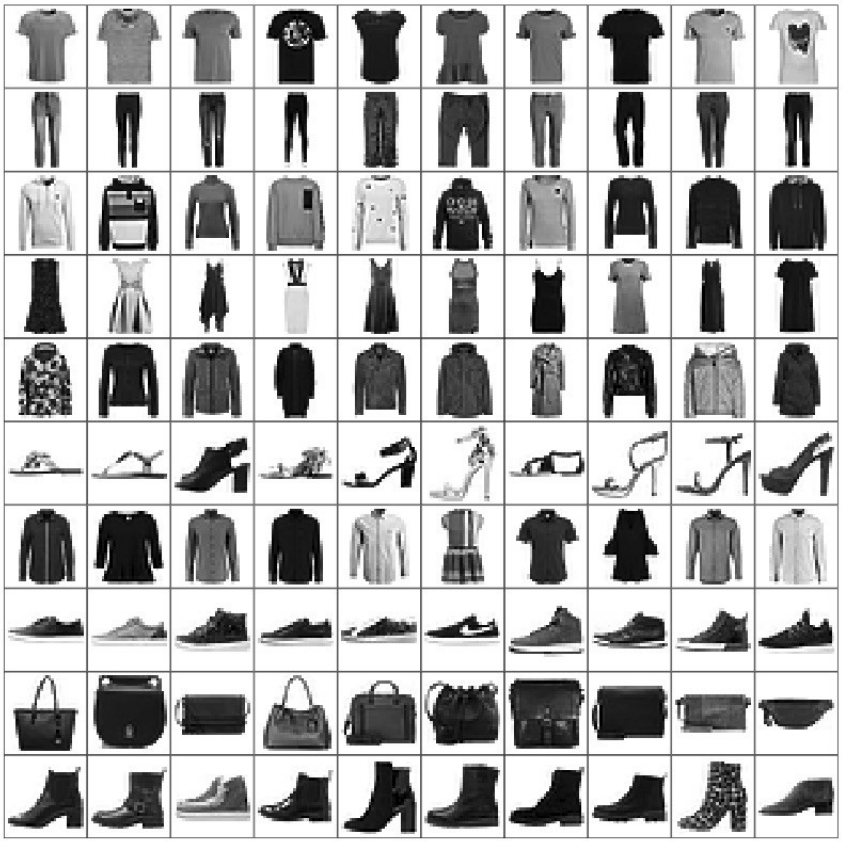


Рисунок 1. Пример изображений Fashion MNIST

Для начала работы откройте Google Colab.

Выполним загрузку необходимых библиотек.

Numpy – это расширение языка Python, добавляющее поддержку больших многомерных массивов и матриц, вместе с большой библиотекой высокоуровневых математических функций для операций с этими массивами.

Matplotlib – библиотека для визуализации данных двумерной и трёхмерной графикой.

```
import keras
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np
```

Загрузим базу изображений MNIST.

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

x_train – массив в который загружаются изображения из обучающей выборки.

y_train – массив в который загружаются целевые данные из обучающей выборки.

x_test – массив в который загружаются изображения из тестовой выборки.

y_test – массив в который загружаются целевые данные из тестовой выборки.

2. Предобработка данных

Для предобработки данных необходимо разделить данные в массивах x_train и x_test на число 255. Это выполняется для того, чтобы привести значения пикселей на изображениях в диапазон [0; 1]. Пиксели на изображениях имеют значения от 0 до 255, а выходные значения нейронной сети имеют значения в диапазоне от 0 до 1. Поэтому, для корректной работы сети мы приводим данные к одному формату.

```
.
.
Допишите код
.
.
```

В следующем блоке кода выполняется преобразование выходных целевых данных нейронной сети к категориальному виду, т.е. целочисленные значения приводятся в двоичный вид.

```
.
.
Допишите код
.
.
```

3. Вывод загруженных изображений на экран

Для того чтобы проверить корректность загрузки наших данных выведем на экран несколько загруженных изображений из датасета. Также выведем класс каждой картинки в виде подписи снизу.

```
plt.figure(figsize=(6,6))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.xticks([])
    plt.yticks([])
```

```
    plt.imshow(x_train[i], cmap=plt.cm.binary)
    plt.xlabel(y_train[i])
```

На рисунке 2 представлены выведенные на экран изображения из обучающей выборки.



Рисунок 2. Изображения из обучающей выборки

4. Реализация топологии нейронной сети

Для решения задачи классификации изображений мы можем использовать библиотеку Keras, которая содержит средства позволяющие реализовать и обучить нейронные сети.

Чтобы реализовать нейронную сеть вставьте и запустите следующий блок кода:

```
from keras.models import Sequential
from keras.layers import *

.
.
Допишите код
.
.
```

Рассмотрим данный код более подробно.

Объект **Sequential** позволяет объявить модель библиотеки Keras. Данная модель позволяет создать нейронную сеть прямого распространения, путем последовательного добавления слоев с помощью метода **add()**.

Flatten преобразует двумерную матрицу (входное изображение) в одномерный массив.

Dense позволяет создать полносвязный слой нейронной сети со следующими параметрами: количество нейронов, **input_shape** – формат входных данных (количество пикселей изображения), **activation** – функция активации.

ReLU – функция активации, выполняющая простое нелинейное преобразование поданных на вход данных (x). Возвращает x, если x > 0 и 0 в противном случае. Отличается высокой скоростью вычисления.

Softmax – функция активации, выполняющая нелинейное преобразование на основе математической функции: взвешенная сигмоидальная функция. Также может быть использована как функция активации для выходного слоя в задачах многоклассовой классификации. Преобразует значения в вектор вероятностей.

Таким образом, вы создаете модель нейронной сети для распознавания 10 классов элементов одежды на изображениях. Пример модели нейронной сети представлена на рисунке 3.

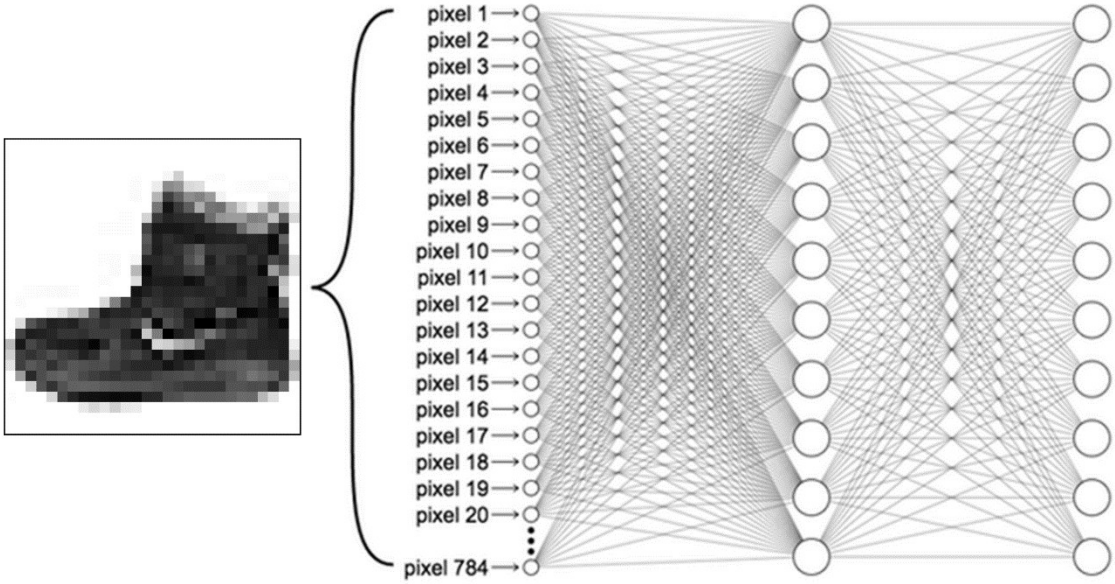


Рисунок 3. Пример модели нейронной сети

Как видно по рисунку изображение размером 28x28 пикселей подаётся на вход полносвязной нейронной сети. Каждый пиксель изображения подаётся на отдельный нейрон входного слоя. Количество нейронов во входном слое нейронной сети 784, что соответствует количеству пикселей на входном изображении. В выходном слое НС расположено 10 нейронов, что соответствует количеству классов распознаваемых объектов.

Далее в процессе работы входные данные распределяются по слоям НС, при этом выполняются операции: умножение на весовые коэффициенты, суммирование данных, вычисление функции активации. В выходном слое происходит классификация путём активации соответствующего нейрона.

5. Подготовка модели к процессу обучения

После того, как Ваша модель была объявлена и собрана вставьте следующие строки кода в Ваш блокнот:

```
from keras.optimizers import Adam


.
.
Допишите код
.
```

```
.
```

Основной принцип Keras – сделать процесс максимально простым, при этом позволяя пользователю полностью его контролировать по мере необходимости.

Из библиотеки Keras импортируем оптимизатор Adam, передав следующие параметры:

Compile – это метод для компиляции нейронной сети, при котором выбираются: функция ошибки (loss), метрики оценки качества по умолчанию (accuracy), оптимизатор (optimizer).

Loss – функция ошибки / функция потерь нейронной сети – математическая дифференцируемая функция, характеризующая разницу между «истинным» значением целевой переменной и предсказанным нейронной сетью значением.

MSE – среднеквадратическая ошибка или среднеквадратичное отклонение (mean squared error, mse).

Metrics – список метрик для вывода (точность 'accuracy' или потери 'loss'), используемых при обучении и тестировании.

Accuracy – точность нейронной сети в процентах.

Optimizer – (функция оптимизатора) - оптимизационный алгоритм, задача которого - изменение весов сети для уменьшения ошибки сети в процессе обучения.

Adam - оптимизационный алгоритм, используемый для обучения нейронной сети, основная функция оптимизатора - изменение весов для уменьшения ошибки сети в процессе обучения. Для каждого нейрона алгоритм изменяет веса индивидуально.

6. Обучение нейронной сети

Для обучения нейронной сети необходимо вызвать метод .**fit()**, используя блок кода ниже:

```
.
.
Допишите код
.
.
```

fit() – метод Keras, запускающий обучение нейронной сети.

Используются следующие параметры:

x_train – входные данные (изображения из обучающей выборки).

y_train – целевые данные (классы входных изображений).

epochs – количество повторений циклов обучения для всей выборки данных.

7. Тестирование работы нейронной сети

Для тестирования нейронной сети необходимо запустить следующий блок кода:

```
.
.
Допишите код
.
.
```

evaluate() – метод оценки модели на тестовых данных.
X_test – входные данные (изображения из тестовой выборки).
y_test_cat – целевые данные (классы входных изображений).

8. Результат работы нейронной сети

Выполните следующий блок кода:

```
.
.
Допишите код
.
.
```

y_pred – выполняет предсказания модели на тестовых данных. Будет содержать вероятности принадлежности каждого примера к каждому классу.

y_pred_classes – преобразует вероятности в предсказанные классы. Для каждого примера выбирается класс с максимальной вероятностью.

y_true – преобразует one-hot метки в числовые классы.

Выведем несколько случайных изображений из тестового датасета. Так же сделаем сверху подписи: зеленый цвет – если нейрсоеть дала правильный результат; красный цвет – если нейросеть ошиблась (рисунок 4).

Для этого создадим фигуру (окно для графиков) размером 12x7 дюймов.

```
plt.figure(figsize=(12, 7))
```

Выбираются 15 случайных индексов из тестового набора (x_test), без повторений (replace=False).

```
random_indices = np.random.choice(len(x_test), size=15, replace=False)
```

Для каждого из 15 индексов выполняется отображение изображения и меток. Создается сетка 3×5 (всего 15 ячеек).

```
for i, idx in enumerate(random_indices):
    plt.subplot(3, 5, i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_test[idx].reshape(28, 28), cmap=plt.cm.binary)
```

Добавление заголовка с метками. Сравниваются предсказание (y_pred_classes) и истинная метка (y_true). Если предсказание верное – текст зеленый, иначе – красный.

```
    predicted_label = y_pred_classes[idx]
    true_label = y_true[idx]
    color = 'green' if predicted_label == true_label else 'red'
    plt.title(f'Pred: {predicted_label}\nTrue: {true_label}', color=color,
fontsize=11)
```

Вывод результата на экран.

```
plt.tight_layout()
plt.show()
```

tight_layout() – предотвращение наложения элементов.
show() – отображение результат.



Рисунок 4. Результат работы нейронной сети

Вывод

В результате выполнения данной лабораторной работы Вы научились решать задачу классификации изображений с элементами одежды Fashion MNIST с помощью нейронных сетей. В рамках данной работы были получены навыки работы с Google Colab, iPython Notebooks и библиотекой машинного обучения Keras.

Критерии оценки

Максимальное число баллов, которое можно получить по лабораторной работе – 8 баллов.

Минимальное число баллов, при котором задание засчитывается – 5 баллов.

Выполнение задания

Выполнено меньше половины задания - 0 баллов.

Большей частью выполнено - 2 балл.

Выполнено полностью - 4 балла.

Качество отчета

Сильные огрехи в оформлении или удовлетворяет менее половины требований к отчету - 0 баллов.

Хорошее оформление. Удовлетворяет большей части требований - 2 балл.
Соответствует всем требованиям - 4 балла.